

```

1  """
2  Starter code for the problem "Cart-pole swing-up with limited actuation".
3
4  Autonomous Systems Lab (ASL), Stanford University
5  """
6
7  from functools import partial
8
9  from animations import animate_cartpole
10
11 import cvxpy as cvx
12
13 import jax
14 import jax.numpy as jnp
15
16 import matplotlib.pyplot as plt
17
18 import numpy as np
19
20 from tqdm import tqdm
21
22
23 @partial(jax.jit, static_argnums=(0,))
24 @partial(jax.vmap, in_axes=(None, 0, 0))
25 def affinize(f, s, u):
26     """Affinize the function `f(s, u)` around `(s, u)`.

```

```

71     The terminal state cost matrix (2-D).
72     Q : numpy.ndarray
73     The state stage cost matrix (2-D).
74     R : numpy.ndarray
75     The control stage cost matrix (2-D).
76     u_max : float
77     The bound defining the control set `[-u_max, u_max]`.
78     ρ : float
79     Trust region radius.
80     eps : float
81     Termination threshold for SCP.
82     max_iters : int
83     Maximum number of SCP iterations.
84
85     Returns
86     -----
87     s : numpy.ndarray
88         A 2-D array where `s[k]` is the open-loop state at time step `k`,
89         for `k = 0, 1, ..., N-1`
90     u : numpy.ndarray
91         A 2-D array where `u[k]` is the open-loop state at time step `k`,
92         for `k = 0, 1, ..., N-1`
93     J : numpy.ndarray
94         A 1-D array where `J[i]` is the SCP sub-problem cost after the i-th
95         iteration, for `i = 0, 1, ..., (iteration when convergence occurred)`
96     """
97     n = Q.shape[0] # state dimension
98     m = R.shape[0] # control dimension
99
100    # Initialize dynamically feasible nominal trajectories
101    u = np.zeros((N, m))
102    s = np.zeros((N + 1, n))
103    s[0] = s0
104    for k in range(N):
105        s[k + 1] = fd(s[k], u[k])
106
107    # Do SCP until convergence or maximum number of iterations is reached
108    converged = False
109    J = np.zeros(max_iters + 1)
110    J[0] = np.inf
111    for i in (prog_bar := tqdm(range(max_iters))):
112        s, u, J[i + 1] = scp_iteration(f, s0, s_goal, s, u, N, P, Q, R, u_max, ρ)
113        dJ = np.abs(J[i + 1] - J[i])
114        prog_bar.set_postfix({"objective change": "{:.5f}".format(dJ)})
115        if dJ < eps:
116            converged = True
117            print("SCP converged after {} iterations.".format(i))
118            break
119    if not converged:
120        raise RuntimeError("SCP did not converge!")
121    J = J[1 : i + 1]
122    return s, u, J
123
124
125 def scp_iteration(f, s0, s_goal, s_prev, u_prev, N, P, Q, R, u_max, ρ):
126     """Solve a single SCP sub-problem for the cart-pole swing-up problem.
127
128     Arguments
129     -----
130     f : callable
131         A function describing the discrete-time dynamics, such that
132         `s[k+1] = f(s[k], u[k])`.
133     s0 : numpy.ndarray
134         The initial state (1-D).
135     s_goal : numpy.ndarray
136         The goal state (1-D).
137     s_prev : numpy.ndarray
138         The state trajectory around which the problem is convexified (2-D).
139     u_prev : numpy.ndarray
140         The control trajectory around which the problem is convexified (2-D).
141     N : int
142         The time horizon of the LQR cost function.
143     P : numpy.ndarray
144         The terminal state cost matrix (2-D).

```

```

145 Q : numpy.ndarray
146     The state stage cost matrix (2-D).
147 R : numpy.ndarray
148     The control stage cost matrix (2-D).
149 u_max : float
150     The bound defining the control set `[-u_max, u_max]`.
151 ρ : float
152     Trust region radius.
153
154 Returns
155 -----
156 s : numpy.ndarray
157     A 2-D array where `s[k]` is the open-loop state at time step `k`,
158     for `k = 0, 1, ..., N-1`
159 u : numpy.ndarray
160     A 2-D array where `u[k]` is the open-loop state at time step `k`,
161     for `k = 0, 1, ..., N-1`
162 J : float
163     The SCP sub-problem cost.
164 """
165 A, B, c = affinize(f, s_prev[:-1], u_prev)
166 A, B, c = np.array(A), np.array(B), np.array(c)
167 n = Q.shape[0]
168 m = R.shape[0]
169 s_cvx = cvx.Variable((N + 1, n))
170 u_cvx = cvx.Variable((N, m))
171
172 # PART (c) #####
173 # INSTRUCTIONS: Construct the convex SCP sub-problem.
174 objective = 0.0
175 constraints = []
176 constraints += [s_cvx[0] == s0]
177 constraints += [cvx.norm(s_cvx[k] - s_prev[k], 'inf') <= ρ for k in range(N+1)]
178 constraints += [cvx.norm(u_cvx[k] - u_prev[k], 'inf') <= ρ for k in range(N)]
179 constraints += [cvx.norm(u_cvx[k], 'inf') <= u_max for k in range(N)]
180 constraints += [s_cvx[k + 1] == A[k] @ s_cvx[k] + B[k] @ u_cvx[k] + c[k] for k in range(N)]
181 objective += cvx.quad_form(s_cvx[N] - s_goal, P)
182 objective += cvx.sum([cvx.quad_form(s_cvx[k] - s_goal, Q) + cvx.quad_form(u_cvx[k], R) for k in range(N)])
183 # END PART (c) #####
184
185 prob = cvx.Problem(cvx.Minimize(objective), constraints)
186 prob.solve()
187 if prob.status != "optimal":
188     raise RuntimeError("SCP solve failed. Problem status: " + prob.status)
189 s = s_cvx.value
190 u = u_cvx.value
191 J = prob.objective.value
192 return s, u, J
193
194
195 def cartpole(s, u):
196     """Compute the cart-pole state derivative."""
197     mp = 1.0 # pendulum mass
198     mc = 4.0 # cart mass
199     L = 1.0 # pendulum length
200     g = 9.81 # gravitational acceleration
201
202     x, θ, dx, dθ = s
203     sinθ, cosθ = jnp.sin(θ), jnp.cos(θ)
204     h = mc + mp * (sinθ**2)
205     ds = jnp.array(
206         [
207             dx,
208             dθ,
209             (mp * sinθ * (L * (dθ**2) + g * cosθ) + u[0]) / h,
210             -((mc + mp) * g * sinθ + mp * L * (dθ**2) * sinθ * cosθ + u[0] * cosθ)
211             / (h * L),
212         ]
213     )
214     return ds
215
216
217 def discretize(f, dt):
218     """Discretize continuous-time dynamics `f` via Runge-Kutta integration."""

```

```

219
220     def integrator(s, u, dt=dt):
221         k1 = dt * f(s, u)
222         k2 = dt * f(s + k1 / 2, u)
223         k3 = dt * f(s + k2 / 2, u)
224         k4 = dt * f(s + k3, u)
225         return s + (k1 + 2 * k2 + 2 * k3 + k4) / 6
226
227     return integrator
228
229
230 # Define constants
231 n = 4 # state dimension
232 m = 1 # control dimension
233 s_goal = np.array([0, np.pi, 0, 0]) # desired upright pendulum state
234 s0 = np.array([0, 0, 0, 0]) # initial downright pendulum state
235 dt = 0.1 # discrete time resolution
236 T = 10.0 # total simulation time
237 P = 1e3 * np.eye(n) # terminal state cost matrix
238 Q = np.diag([1e-2, 1.0, 1e-3, 1e-3]) # state cost matrix
239 R = 1e-3 * np.eye(m) # control cost matrix
240 p = 1.0 # trust region parameter
241 u_max = 8.0 # control effort bound
242 eps = 5e-1 # convergence tolerance
243 max_iters = 100 # maximum number of SCP iterations
244 animate = False # flag for animation
245
246 # Initialize the discrete-time dynamics
247 fd = jax.jit(discretize(cartpole, dt))
248
249 # Solve the swing-up problem with SCP
250 t = np.arange(0.0, T + dt, dt)
251 N = t.size - 1
252 s, u, J = solve_swingup_scp(fd, s0, s_goal, N, P, Q, R, u_max, p, eps, max_iters)
253
254 # Simulate open-loop control
255 for k in range(N):
256     s[k + 1] = fd(s[k], u[k])
257
258 # Plot state and control trajectories
259 fig, ax = plt.subplots(1, n + m, dpi=150, figsize=(15, 2))
260 plt.subplots_adjust(wspace=0.45)
261 labels_s = (r"$x(t)$", r"$\theta(t)$", r"$\dot{x}(t)$", r"$\dot{\theta}(t)$")
262 labels_u = (r"$u(t)$",)
263 for i in range(n):
264     ax[i].plot(t, s[:, i])
265     ax[i].axhline(s_goal[i], linestyle="--", color="tab:orange")
266     ax[i].set_xlabel(r"$t$")
267     ax[i].set_ylabel(labels_s[i])
268 for i in range(m):
269     ax[n + i].plot(t[:-1], u[:, i])
270     ax[n + i].axhline(u_max, linestyle="--", color="tab:orange")
271     ax[n + i].axhline(-u_max, linestyle="--", color="tab:orange")
272     ax[n + i].set_xlabel(r"$t$")
273     ax[n + i].set_ylabel(labels_u[i])
274 plt.savefig("cartpole_swingup_constrained.png", bbox_inches="tight")
275
276 # Plot cost history over SCP iterations
277 fig, ax = plt.subplots(1, 1, dpi=150, figsize=(8, 5))
278 ax.semilogy(J)
279 ax.set_xlabel(r"SCP iteration ${i}$")
280 ax.set_ylabel(r"SCP cost $\bar{J}(\bar{x}^{\{i\}}, \bar{u}^{\{i\}})$")
281 plt.savefig("cartpole_swingup_constrained_cost.png", bbox_inches="tight")
282 plt.show()
283
284 # Animate the solution
285 if animate:
286     fig, ani = animate_cartpole(t, s[:, 0], s[:, 1])
287     ani.save("cartpole_swingup_constrained.mp4", writer="ffmpeg")
288     plt.show()

```